

การสร้างชุดทดสอบ Java: ผลรอบที่ 2

กลุ่ม 14 / CP353201 Software Quality Assurance

CMA-ES และ FSCS-ART เปรียบเทียบกับ Claude และ Gemini ผ่าน KCU IntelSphere

ข้อมูล ณ 2026-10-01T09:37:40.475757+00:00 UTC

สถานะ: สำเร็จ 136/204 รอบตามแผนที่งานทดลองยังไม่ครบ และยังไม่ใช้หลักฐานการส่ง Classroom

ขอบเขตและการเปลี่ยนแปลง

อ้างอิงเกณฑ์ SQA_Project_2026.pdf สำหรับการส่งรอบ 2: ใช้สองอัลกอริทึมและสองเครื่องมือ AI สร้าง unit tests ของ Java projects ใน Defects4J เปรียบเทียบ coverage, fault detection และ efficiency พร้อมโค้ด หลักฐาน รายงาน การนำเสนอและ demo

เจ้าของงานกำหนดให้ส่ง prompt ผ่าน <https://gen.ai.kku.ac.th/chat> เท่านั้น และเลือกเฉพาะ Claude กับ Gemini จึงใช้กลุ่มเปรียบเทียบใหม่ CMA-ES, FSCS-ART, KCU Claude และ KCU Gemini ผล Claude

โดยตรงและโมเดลตระกูลอื่นในชุดเดิมไม่รวมในผลหลัก ไม่อ้างว่าผู้สอนอนุมัติการเปลี่ยนเครื่องมือจากแผนรอบแรก

17 projects $\times$ 1 active bug/project $\times$ 3 run indices (101/102/103) $\times$ 4 approaches = 204 รอบ งบสูงสุด 30 วิธีทดสอบต่อชุด AI ส่วนอัลกอริทึมเสนอ 30 input vectors จำนวนนี้เป็นแผนที่งาน ไม่ใช่จำนวนที่โจทย์ระบุโดยตรง หนึ่ง bug ต่อ project ไม่ครอบคลุมทุก bug ในฐานข้อมูล

โปรเจกต์: Chart, Cli, Closure, Codec, Collections, Compress, Csv, Gson, JacksonCore, JacksonDatabind, JacksonXml, Jsoup, JXPath, Lang, Math, Mockito, Time

สถานะและจำนวนชุดทดสอบ

วิธี	สำเร็จ/51	โปรเจกต์/17	Test methods	ใช้เดิม	ทำใหม่	ขาด	ยังไม่สำเร็จ
CMA-ES	51/51	17/17	1,529	51	0	0	0
FSCS-ART	51/51	17/17	1,529	51	0	0	0
KCU Claude	5/51	5/17	114	3	2	41	5
KCU Gemini	29/51	17/17	564	13	16	20	2

จำนวน methods เป็นผลรวม declared methods ใน completed suites อาจมีสถานการณ์ซ้ำข้ามรอบ ไม่ใช่ unique scenarios และไม่คูณจำนวนการรัน fixed/buggy ผล failed/missing มีค่าการวัดที่ไม่มีเป็น null ไม่แทนด้วยศูนย์

มี fresh completed records ที่ซ้ำ identity กับผลเดิม 1 รายการ: Time/Gemini/102 เก็บผลใหม่เป็น secondary validation และคงผลเดิมเป็น primary ตามลำดับที่มีหลักฐานก่อน โดยไม่ใช่ fault/coverage เลือกผล จึงไม่เพิ่มจำนวนรอบหรือ methods ในตาราง primary audit ตรวจผลใหม่ที่สำเร็จ 17 Gemini records รวม secondary หนึ่งรายการ และ 2 Claude records

ผลเดิมผ่าน baseline audit 171/171 records การนำมาใช้ในชุดนี้อ้าง record path และ SHA-256

ไม่แก้ไขโมเดลหรือ generator เดิม เลือกเฉพาะผลอัลกอริทึมและคำตอบ KCU ที่ระบุ resolved model เป็น Claude/Gemini และ metadata ตรงกัน ผล KCU เดิมบางรายการมาจาก Auto Router ซึ่งแยกจากคำตอบใหม่ที่เลือก agent โดยตรง

วิธีสร้างอินพุตและ oracle

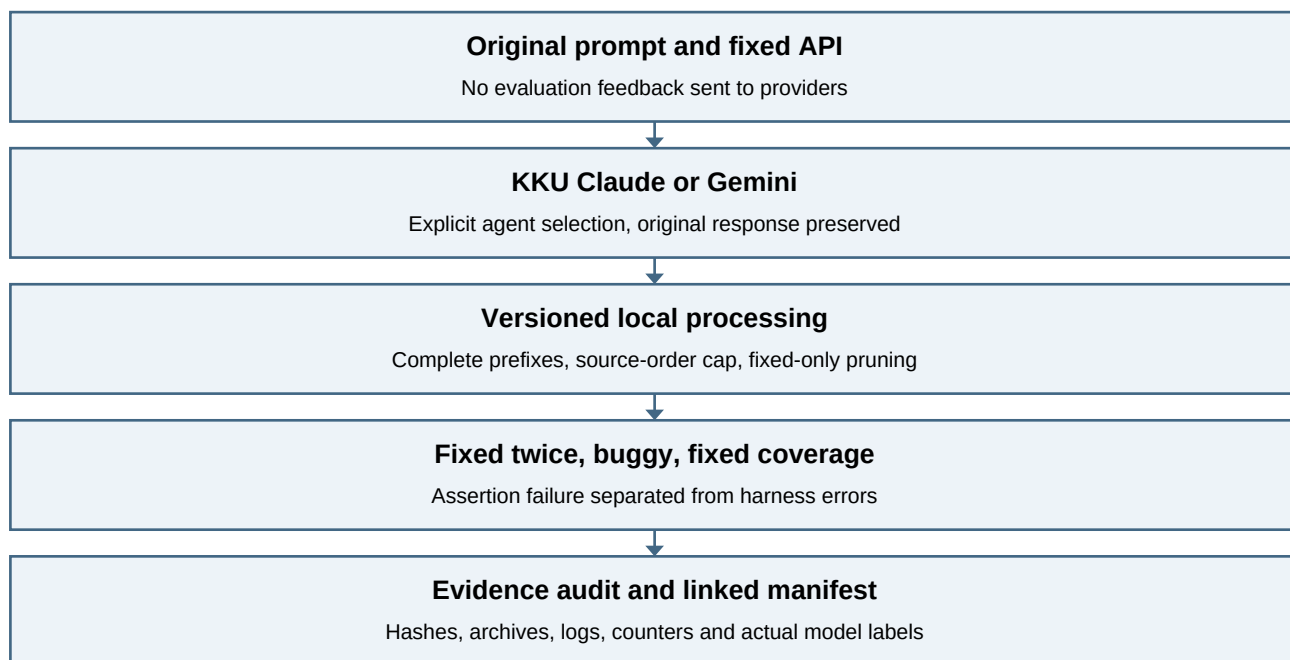
FSCS-ART เลือก candidate ที่มีระยะห่างต่ำสุดจากอินพุตก่อนหน้านี้มากที่สุดใน normalized input space ทำให้ชุดอินพุตกระจายตัว CMA-ES ปรับ mean, covariance และ step size ของการค้นหาตัวแปรต่อเนื่อง ตัว implementation ในงานนี้ใช้ความถี่ของ fixed behavior เป็น objective เพื่อเพิ่มความหลากหลายของ output ไม่ใช่ coverage เป็น fitness

SqaProbe ใช้ signatures ที่มีทั้ง fixed และ buggy revisions รวมสมาชิก non-public ที่ reflection เข้าถึงได้ สร้าง scalar, enum, string, array, collection และ constructor fixture แบบจำกัด ทุก proposed input สังเกต fixed สองครั้ง เก็บเฉพาะผลคงที่และ input ไม่เข้ามาสร้าง assertions ผล opaque object ยืนยันได้เพียง runtime type ส่วน serialized output ยาวเกิน 16000 ตัวอักษรใช้ SHA-256 และ byte length ข้อจำกัดนี้ลดการทดสอบ object graphs และ stateful sequences

AI ใช้ prompt ต้นฉบับของแต่ละ project ใน ai-context/prompt.md ให้ fixed source/build/API context ส่งครั้งเดียวต่อ run index ไม่ส่ง compile/error/coverage/buggy logs กลับให้ AI โค้ดจากส่วน thinking ไม่ถือเป็น final source

เก็บ raw DOM-rendered response และ code blocks, model label จริง, เวลาเริ่มและเวลาสังเกตคำตอบ, URL และ screenshot ไม่อ้าง hidden temperature, token count หรือ resolved version ของ latest AI run indices เป็นหมายเลขรอบ ไม่ใช่ RNG seeds ของโมเดล

ขั้นตอนประเมินและหลักฐาน



สร้าง external JUnit suite ตามรุ่นที่ project รองรับ ประเมิน fixed ครั้งที่ 1 และ fixed ครั้งที่ 2 แล้วรัน buggy และเก็บ Cobertura coverage บน fixed นิยาม fault detected คือ assertion ผ่าน fixed ซ้ำแต่ล้มเหลวบน buggy โดยแยก compiler, harness และ timeout errors ออก อ่าน triggering test ประกอบก่อนสรุป semantic defect

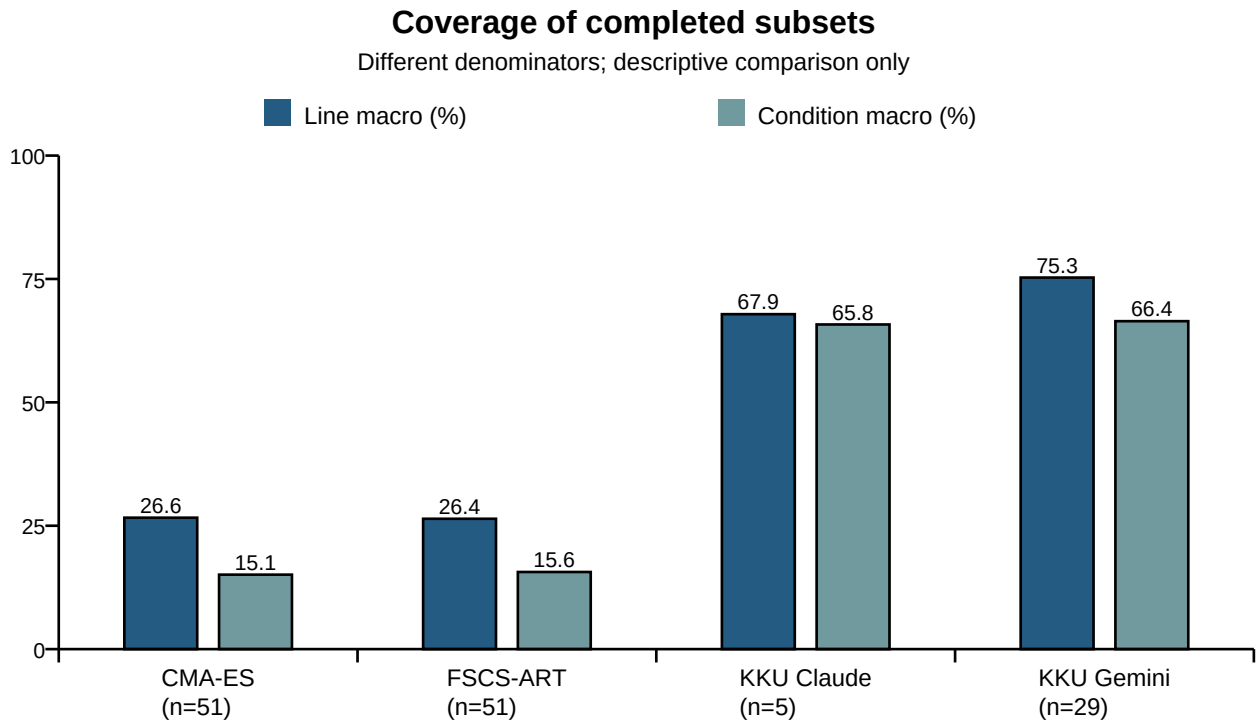
Coverage วัดเฉพาะ classes ที่แก้ไขเพื่อซ่อม bug ไม่ใช่ทุก class ใน project เก็บ covered/total counters, coverage.xml, command.json/log, failing_tests, suite archive และ processing snapshots ที่ตรวจ hash ได้

เครื่องใหม่ใช้ Ubuntu WSL, Java 11, Defects4J 3.0.1 commit

6d54320e0db5a357f9ab38a8e4d2e5aead7e1c09 ผล fixed source ของทั้ง 17 projects ตรวจ SHA ตรง

ai-context เดิม ดู environment receipts สำหรับ versions/checkouts ผล baseline มาจากอีกเครื่อง จึงมีผลของ hardware, host load, caches และ build overhead ต่อเวลา

RQ1: Coverage ของชุดที่สำเร็จ



Macro = ค่าเฉลี่ย covered/total ต่อ run; micro = ผลรวม covered / ผลรวม total ไม่ใช่ union coverage และนับโค้ดซ้ำข้ามรอบ ตัวหารและ successful subsets ต่างกัน จึงใช้ผลต่อไปนี่เชิงพรรณนา

วิธี	n	Line macro	Condition macro	Line micro
CMA-ES	51	26.6%	15.1%	21.3%
FSCS-ART	51	26.4%	15.6%	21.6%
KKU Claude	5	67.9%	65.8%	27.4%
KKU Gemini	29	75.3%	66.4%	61.6%

CMA-ES และ FSCS-ART มีครบ 51 รอบบน sample เดียวกัน Line macro ต่างกัน 0.22 percentage points ผลใกล้เคียงกันในค่าเฉลี่ย แต่ coverage ต่ำในบาง fixtures และแต่ละ project มีขนาด classes ต่างกัน การเทียบ AI ต้องดูตัวหารและกลุ่มตรงกันแยก เพราะ sample ที่สำเร็จมี composition ต่างจากอัลกอริทึม

RQ2: การตรวจพบ sampled bugs

วิธี	พบ/รอบที่วัดได้	พบ/bugs ที่วัดได้	อัตราระดับ bug
CMA-ES	9/51	5/17	29.4%
FSCS-ART	10/51	4/17	23.5%
KKU Claude	3/5	3/5	60.0%
KKU Gemini	14/29	9/17	52.9%

CMA-ES พบ 5/17 sampled bugs และ FSCS-ART พบ 4/17 แม้ FSCS-ART มีรอบที่พบ fault มากกว่า (10 เทียบกับ 9) การนับระดับ run จึงตอบต่างจากระดับ distinct bug และไม่ควรเลือกวิธีจากตัวเลขใดตัวเลขหนึ่งโดยไม่ระบุหน่วย

ตรวจพบซ้ำหลาย methods/runs ของ project-bug เดียวกันนับเป็นหนึ่ง bug ในอัตราระดับ bug การไม่ตรวจพบ sampled bug ไม่ยืนยันว่า suite ไม่มีประโยชน์ หรือ project ไม่มีข้อผิดพลาด

เปรียบเทียบวิธีในกลุ่มตรงกัน

มี 4 project-bug-index groups ที่ครบทั้งสี่วิธี: Chart-1/s101, Collections-1/s101, Csv-1/s101, Jsoup-1/s101

วิธี	n กลุ่มตรงกัน	Line macro	Condition macro	พบ/รอบ
CMA-ES	4	27.1%	16.6%	0/4
FSCS-ART	4	27.5%	16.3%	0/4
KKU Claude	4	59.8%	57.2%	2/4
KKU Gemini	4	74.0%	70.8%	2/4

กลุ่มตรงกันลดความต่างของ project/index แต่จำนวนกลุ่มน้อยและเลือกจากชุดที่ผ่านทั้งหมด ยังมี selection bias, model/version และ processing effort ที่ต่างกัน ไม่ใช่เป็น causal ranking และไม่ทดสอบนัยสำคัญกับ sample นี้

RQ3: Efficiency และความสำเร็จของ workflow

วิธี	Compile ผ่าน/วัด	median evaluation (s)	median generation (s)
CMA-ES	51/51	55.64	13.50
FSCS-ART	51/51	54.42	13.61
KKU Claude	5/5	45.66	165.00
KKU Gemini	29/29	36.25	105.70

Compile rate ใช้เฉพาะ run ที่มีผล compile ชัดเจน ไม่รวม provider failures ที่ไม่ได้คอมไพล์ และไม่รวม attempt ที่ถูกเก็บเป็น history ก่อนซ่อม ค่า generation ของ AI เป็น submit-to-observed-completion upper bound รวมการรอผู้ปฏิบัติงาน ค่าอัลกอริทึมเป็นเวลาเครื่องวัด จึงเปรียบเทียบความเร็วโดยตรงไม่ได้

evaluation รวม build/JVM/test/coverage overhead และได้ผลของ cache warming งบ 30 proposed inputs กับ 30 AI methods ไม่ใช่ effort ที่เท่ากัน token/cost/model compute ไม่ปรากฏใน UI จึงไม่สร้างตัวเลขขึ้นเอง local repair/pruning ต้องดู history เพิ่ม ไม่ใช่เพียง final-run เวลา

การซ่อมและต้นทุนที่เปิดเผย

ใช้ source-order cap 30 methods, UI suffix cleanup และ fixed-only pruning สูงสุดสองรอบตาม processor ที่เก็บ hash ถ้าต้องเพิ่ม compatibility ใช้เวอร์ชันใหม่ เก็บ raw source และ failed attempts เดิม ห้ามแก้ expected values จาก buggy outcomes

เวอร์ชันใหม่ v30 กู้ complete prefix ของ Gemini Lang; v31 แก้ overload ambiguity ของ Time ด้วย (String)null โดยคง assertion; v32 กู้ complete prefix ของ Claude Jsoup; v33 กู้ Claude Cli; v34 กู้ Gemini Codec; v35 cast Collections iterator เป็น ResettableIterator; v36/v37 unbox Integer sibling index สองตำแหน่งของ Jsoup ตาม fixed compiler/API ใช้ source hashes ที่ระบุ ไม่อ้างว่าต้นฉบับ AI คอมไพล์ได้โดยไม่มี processing

เรียกผลที่ผ่านว่า AI-assisted with local processing ทุก attempt เก็บก่อน/หลัง policy SHA และ driver/source snapshots มี audit แบบเต็มเฉพาะ completed records และ audit provenance แยกตรวจ provider/failure/manifest ไม่มีการรับรองคะแนนจากผู้สอน

โมเดลและที่มาของแต่ละกลุ่ม

วิธี	label จริง	ที่มา	records
KKU Claude	claude-sonnet-5	reused-audited-baseline	3
KKU Claude	claude-sonnet-latest	new-kku-capture	7
KKU Gemini	gemini-3.6-flash	reused-audited-baseline	6
KKU Gemini	gemini-flash	new-kku-capture	18
KKU Gemini	gemini-pro	reused-audited-baseline	7

ตารางนับ records ที่มี label รวม failed records ส่วน metric tables นับเฉพาะ completed การรวมตระกูลโมเดลต่าง versions ใช้ตามข้อจำกัดบริการและแยกเปิดเผย ไม่อ้างว่าเป็นโมเดลเดียวกันหรือ independent deterministic repetitions

อุปสรรคและบทเรียน

บริการ Claude ใน KCU แสดง daily usage 100% ระหว่างเก็บ Compress รอบ 101 และ Gemini แสดง 100% หลังเก็บ Jsoup รอบ 103 ไม่ปรากฏ reset time ที่ยืนยันได้ เก็บ quota status และ screenshots จริง ไม่สร้างผลทดแทนจาก AI อื่น Claude Lang/Time มี server busy และ Math ไม่มี final code ที่สังเกตได้ Mockito และ Gemini Cli ดัดก่อนมี test method สมบูรณ์ จึงบันทึก generation_failed

ค่าตอนถูกตัดกลาง Java, overloaded APIs และ fixture differences ทำให้ raw output ใช้ไม่ได้ทันที การตรวจ parser/fixed ซ้ำและการเก็บ version history ช่วยกุส่วที่ใช้ได้โดยไม่ปิดบังปัญหา Coverage สูงยังไม่รับประกัน fault detection: อ่านตัวอย่าง Jsoup ใน demo-guide-kku-only.md และเปรียบเทียบ assertions กับ failing_tests

Operator incident: Math/102 exports ถูกบันทึกไป Lang/102 ชั่วคราวหลัง Lang ประเมินเสร็จ ัก raw Lang response จาก evaluator snapshot และเรียกแซท Lang เดิมกลับมาได้ SHA ตรงทั้ง response และ Java source ภาพ/DOM exports เป็นการสังเกตใหม่ ไม่อ้างว่าเป็นภาพเดิม เก็บไฟล์ที่บันทึกผิดและ recovery receipt ใน results/validation/operator-capture-routing-20261001 Math ประเมินจาก request-start และ capture ที่แก้ routing ก่อนประเมิน

ภาคผนวก: ผลต่อโปรเจกต์

ตารางแสดงจำนวน completed runs / 3 ตัวอย่าง ให้ดู pending-runs.csv สำหรับ index และเหตุผลที่ยังขาด

Project	CMA-ES	FSCS-ART	KKU Claude	KKU Gemini
Chart	3/3	3/3	1/3	3/3
Cli	3/3	3/3	1/3	1/3
Closure	3/3	3/3	0/3	1/3
Codec	3/3	3/3	0/3	1/3
Collections	3/3	3/3	1/3	2/3
Compress	3/3	3/3	0/3	1/3
Csv	3/3	3/3	1/3	3/3
Gson	3/3	3/3	0/3	1/3
JacksonCore	3/3	3/3	0/3	1/3
JacksonDataBind	3/3	3/3	0/3	2/3
JacksonXml	3/3	3/3	0/3	1/3
Jsoup	3/3	3/3	1/3	3/3
JXPath	3/3	3/3	0/3	1/3
Lang	3/3	3/3	0/3	2/3
Math	3/3	3/3	0/3	2/3
Mockito	3/3	3/3	0/3	2/3
Time	3/3	3/3	0/3	2/3

แหล่งข้อมูลและการทำซ้ำ

ผลลัพธ์: output/kku-only-20261001/summary.json, analysis.json, study-manifest.csv และ record.json ตาม paths ใน manifest ตรวจสอบ provenance-audit.json และ baseline/claude/gemini-evidence-audit.json ก่อนใช้ ทุก completed result ต้องผ่าน audit ที่มี record hash ตรงกัน

การทำซ้ำ: python3 scripts/study/kku_only.py status แล้วรัน audit_kku_provenance.py สำหรับผลใหม่ใช้ evaluate_provider_normalized_vN.py ที่ตรง processing-policy-vN.json และ runtime paths ของเครื่องนี้ เก็บคำตอบผ่านหน้า KKU ด้วย exact original prompt ตาม docs/KKU_ONLY_CONTINUATION.md

เอกสารงาน: docs/reference/SQA_Project_2026.pdf แผน/วิธีเดิม: docs/study-protocol.md และ scripts/study/generate.py ข้อจำกัด AI ปัจจุบัน: results/study/kku-only-20261001/protocol.json

Hansen, N. (2016, revised 2023). The CMA Evolution Strategy: A Tutorial.
<https://arxiv.org/abs/1604.00772>

Chen, T. Y., Leung, H., & Mak, I. K. (2004). Adaptive Random Testing. ASIAN 2004, 320–329.
https://doi.org/10.1007/978-3-540-30502-6_23

Defects4J 3.0.1 official release documentation: docs/reference/defects4j-readme-v3.0.1.md และ
<https://github.com/rjust/defects4j/tree/v3.0.1>

ผู้จัดทำ

นายธนินธร อันทมนตร 673380043-6 / นายศุภกร กรมรินทร์ 673380061-4 / นายณัฏฐพล เพ็งพล 673380267-4 /
นายณัฐกรรณ์ อินธิสาร 673380268-2

รายชื่อจากรายงานรอบแรกของกลุ่ม 14 สถานะ GitHub/Classroom ให้ตรวจสอบ delivery-status.json
ไม่ได้คิดว่าไฟล์ที่สร้างในเครื่องคือการส่งงานแล้ว